MANUAL TÉCNICO

Sistema de Red de Transporte Urbano

Autocompletado, búsqueda directa, grafos ponderados y análisis de conectividad

Proyecto No.02: Proyecto Red Transporte Urbano

Lenguaje C# sobre .NET 8, interfaz Windows Forms

CHOLOTÍO MENDOZA CARLOS ANDRÉS - 1517925
(Autocompletado y búsqueda directa)
DE LEÓN GONZÁLEZ ALEJANDRO DANIEL – 1502425
(Grafos y algoritmos de caminos óptimos)
AGUILÓN CARDONA ANDRES SEBASTIÁN - 1541025
(Union-Find y persistencia de datos)

Quetzaltenango, 5 de noviembre de 2026

Contenido

1. Introducción **3**
1.1 Propósito del documento 3
1.2 Alcance del sistema 3
1.3 Restricciones de implementación 3
1.4 Notación empleada en el análisis de complejidad 3
2. Descripción general del sistema **4**
2.1 Organización por módulos 4
2.2 Flujo unificado de trabajo 4
2.3 Entorno técnico y archivos del proyecto 4
3. Datos de entrada **5**
3.1 Archivo paradas.csv 5
3.2 Archivo conexiones.csv 5
4. Diagrama de clases **5**
4.1 Descripción de las clases 9
5. Diagramas de flujo **9**
6. Explicación de la implementación **13**
6.1 Clase Parada 13
6.2 Árbol de prefijos (Trie) 13
6.3 Tabla Hash 13
6.4 Grafo y algoritmos de caminos óptimos 14
6.5 Union-Find 15
6.6 Lector de archivos CSV 16
6.7 Interfaz gráfica 16
7. Análisis de complejidad (Big-O) **17**
7.1 Resumen de complejidades 17
7.2 Justificación de las estructuras de búsqueda 18
7.3 Justificación del grafo y de los algoritmos de rutas 19
7.4 Justificación de Union-Find 21
7.5 Costo de una consulta de ruta completa 21
8. Limitaciones conocidas y mejoras propuestas **21**
9. Conclusiones **22**

1. Introducción

1.1 Propósito del documento

El presente manual describe el diseño y la implementación del sistema de Red de Transporte Urbano, una aplicación de escritorio desarrollada en C# sobre .NET 8 con Windows Forms. El documento recoge la arquitectura general, el diagrama de clases, los diagramas de flujo de los procesos principales, la explicación detallada de cada estructura de datos y algoritmo, y el análisis de complejidad asintótica (Big-O) de las operaciones. Cada estructura se justifica frente a las alternativas que se consideraron durante el diseño.

1.2 Alcance del sistema

La aplicación permite realizar las siguientes operaciones sobre una red de paradas de transporte:

- Cargar desde archivos CSV el catálogo de paradas y las conexiones entre ellas.
- Buscar paradas por prefijo de su nombre con autocompletado. • Recuperar los datos de una parada a partir de su código único.
- Verificar si dos paradas pertenecen a una misma zona conectada de la red.
- Calcular la ruta de menor costo entre un origen y un destino mediante el algoritmo de Dijkstra.
- Explorar la red a partir de una parada con recorridos en anchura (BFS) y en profundidad (DFS).

1.3 Restricciones de implementación

De acuerdo con las restricciones del proyecto, las estructuras de datos se implementan de forma manual sobre arreglos nativos y nodos enlazados, sin recurrir a colecciones genéricas como List<T> o Dictionary<K,V>. La única excepción es la cola Queue<T>, que se emplea exclusivamente en el recorrido BFS y requiere el espacio de nombres System.Collections.Generic. La lectura de archivos se realiza únicamente con File.ReadAllLines.

1.4 Notación empleada en el análisis de complejidad

Símbolo	Significado
n	Número de paradas registradas (elementos almacenados en el Trie y en la Tabla Hash).
m	Longitud, en caracteres, de un nombre de parada o de un prefijo de búsqueda.
k	Cantidad de sugerencias solicitadas. La interfaz limita este valor a 10.
L	Longitud media del tramo de un nombre que resta después del prefijo escrito.
M	Número de cubos de la Tabla Hash (101).
V	Número de vértices del grafo (paradas).
E	Número de aristas del grafo (conexiones). En un grafo no dirigido cada arista se almacena como dos arcos.

2. Descripción general del sistema

2.1 Organización por módulos

El sistema se divide en tres módulos funcionales, cada uno a cargo de un integrante del equipo, más un conjunto de elementos compartidos. Todos los tipos residen en el espacio de nombres ProyectoRedTransporte.

Módulo	Clases	Responsabilidad
Integrante 1. Autocompletado y búsqueda directa	NodoTrie, Trie, NodoHash, TablaHash	Sugerir paradas por prefijo del nombre y recuperar el objeto Parada a partir de su código.
Integrante 2. Grafos y algoritmos de caminos óptimos	MinHeap, NodoArista, RutaResultado, Grafo	Representar la red como grafo ponderado, calcular la ruta de menor costo (Dijkstra) y recorrer la red (BFS y DFS).
Integrante 3. Union-Find y persistencia de datos	UnionFind, LectorCSV	Determinar la conectividad entre paradas y cargar los datos desde los archivos CSV.
Elementos compartidos	Parada, FormPrincipal, Program	Entidad de dominio, interfaz gráfica y punto de entrada de la aplicación.

2.2 Flujo unificado de trabajo

La interfaz presenta un flujo en cuatro pasos encadenados, que el propio título de la ventana resume como "Trie, Hash, UnionFind, Grafo". Cada estructura cumple un papel distinto y la salida de una es la entrada de la siguiente:

Paso	Estructura	Función dentro de la consulta
A	Trie	Convierte el prefijo escrito por el usuario en una lista de sugerencias. Cada sugerencia conserva el código único de la parada.
B	Tabla Hash	Con el código seleccionado, localiza en tiempo constante promedio el objeto Parada completo.
C	Union-Find	Verifica si origen y destino pertenecen a la misma zona conectada. Si no es así, se evita ejecutar el algoritmo de rutas.
D	Grafo	Ejecuta Dijkstra para obtener la ruta de menor costo, o BFS y DFS para explorar la red.

2.3 Entorno técnico y archivos del proyecto

El proyecto se compila con el SDK de .NET 8 para Windows (net8.0-windows) como aplicación de ventanas (WinExe) con Windows Forms habilitado. El análisis de referencias nulas está desactivado en el archivo de proyecto. Los archivos paradas.csv y conexiones.csv se copian siempre al directorio de salida, de modo que el ejecutable los encuentre junto a él.

Archivo	Contenido
Estructuras.cs	Clase Parada, Trie, Tabla Hash, MinHeap, Grafo, UnionFind y LectorCSV.
FormPrincipal.cs	Lógica de la interfaz: eventos de botones, autocompletado y ejecución de los algoritmos.
Archivo	**Contenido**
FormPrincipal.Designer.cs	Declaración y configuración visual de los controles (parte partial de la clase).
RedDeTransporte.cs	Clase Program con el método Main, que inicia la aplicación.
MIO.csproj	Configuración del proyecto y regla de copia de los archivos CSV.
paradas.csv / conexiones.csv	Datos de entrada del sistema.

3. Datos de entrada

3.1 Archivo paradas.csv

Cada línea describe una parada con cuatro campos separados por coma: código, nombre, latitud y longitud. La primera línea de encabezado se omite cuando comienza con la palabra "codigo". Los números se interpretan con la cultura invariante, por lo que el separador decimal es el punto. Las líneas vacías, las que tienen menos de cuatro campos y las que contienen coordenadas no numéricas se ignoran. Como el separador es la coma, los nombres de parada no pueden contenerla.

3.2 Archivo conexiones.csv

Cada línea define una conexión entre dos paradas: código de origen, código de destino y, de forma opcional, el peso. Si el peso no se proporciona, el grafo calcula la distancia geográfica entre ambas paradas con la fórmula de Haversine, en kilómetros. Las conexiones que mencionan códigos inexistentes se descartan. El grafo se crea como no dirigido, de modo que cada línea genera la conexión en ambos sentidos.

4. Diagrama de clases

El diagrama de clases se presenta en tres figuras para conservar la legibilidad. La Figura 1 muestra las estructuras de búsqueda, la Figura 2 el grafo con sus clases auxiliares y la Figura 3 la capa de aplicación que integra todas las estructuras. Se emplea la notación UML: el rombo relleno indica composición, la flecha punteada indica dependencia y la flecha simple indica asociación.

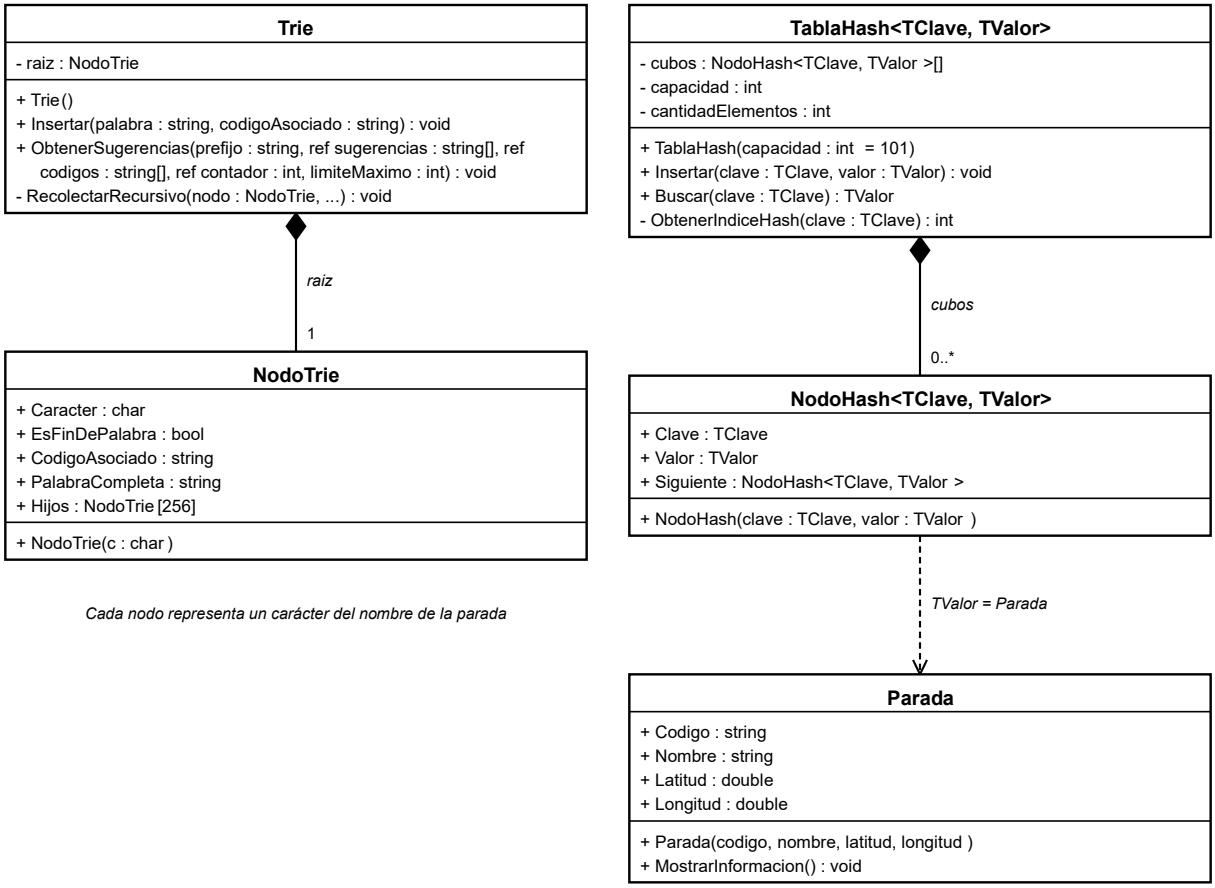


Figura 1. Diagrama de clases de las estructuras de búsqueda (Trie, Tabla Hash y Parada).

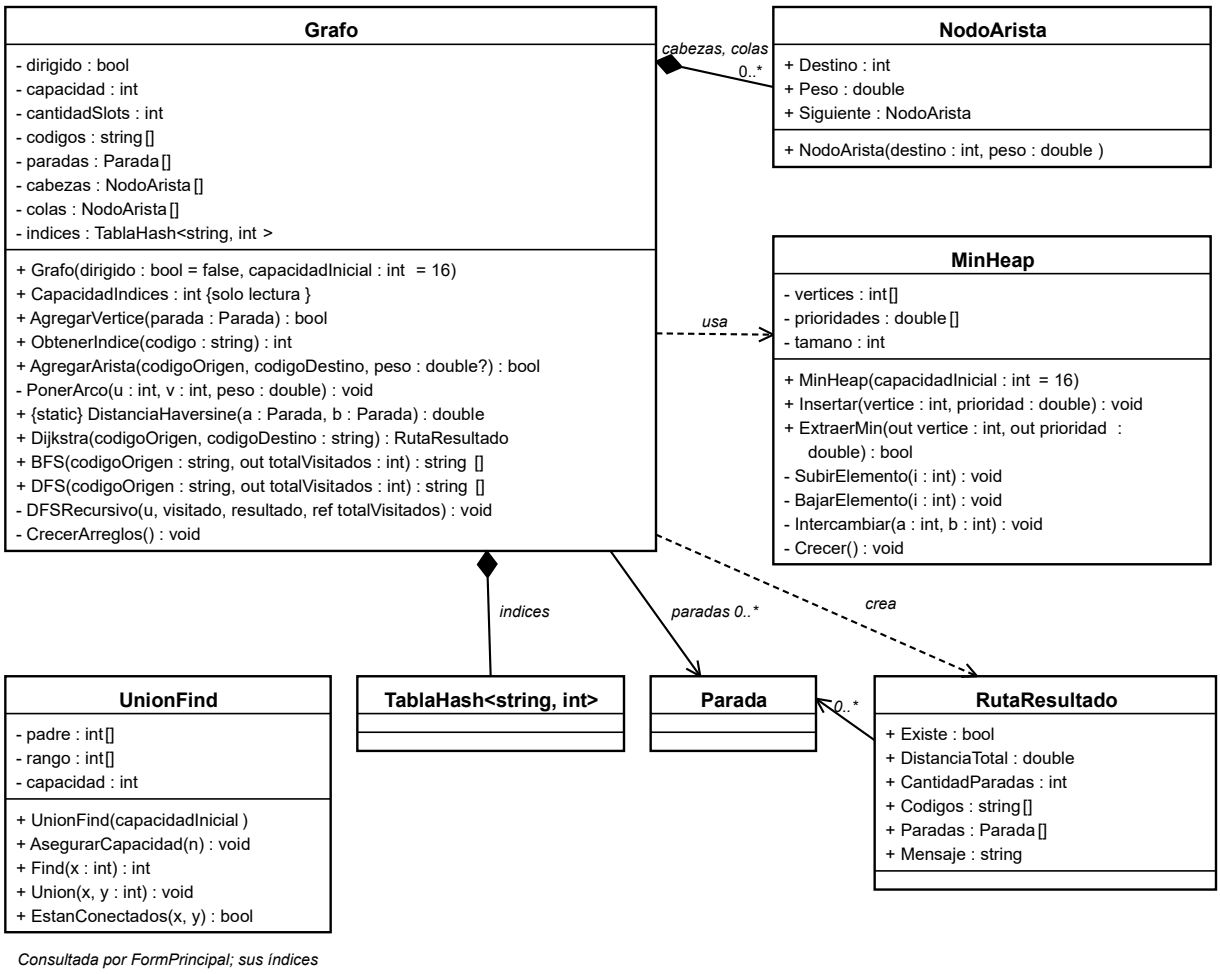


Figura 2. Diagrama de clases del grafo, sus clases auxiliares y la estructura Union-Find.

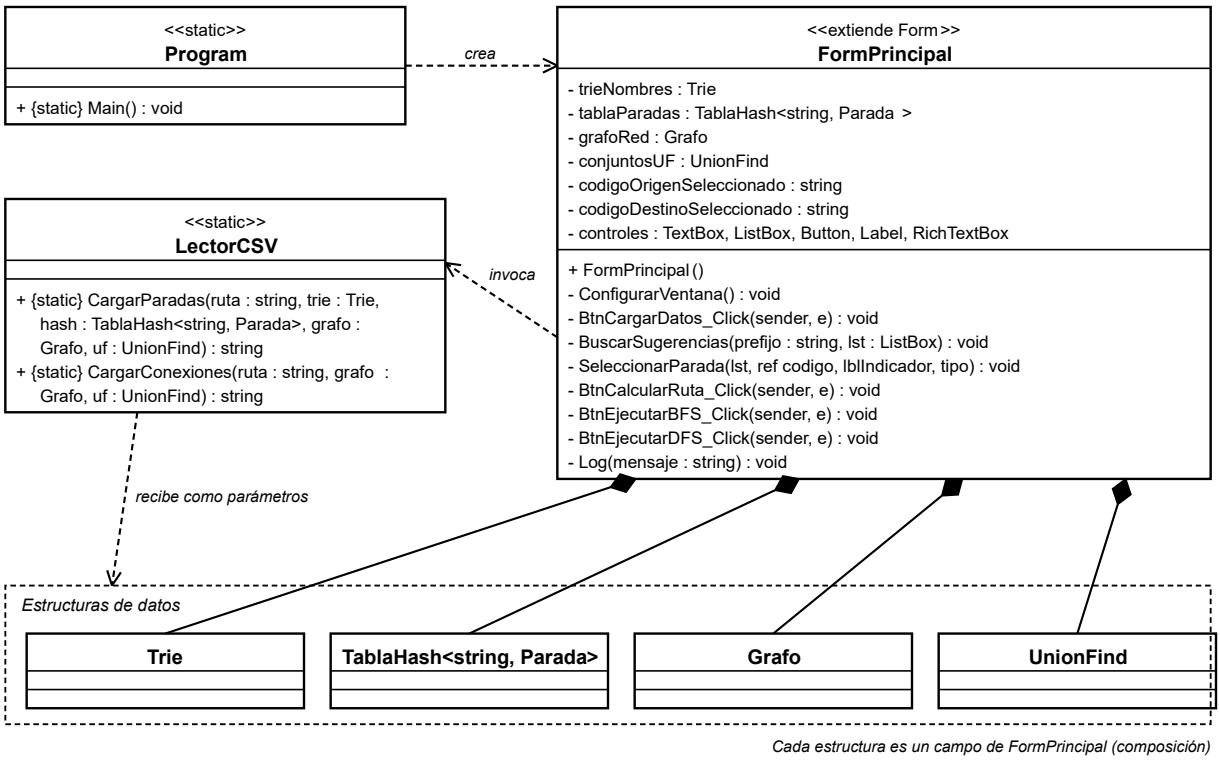


Figura 3. Diagrama de clases de la capa de aplicación.

4.1 Descripción de las clases

Clase	Responsabilidad
Parada	Entidad de dominio. Almacena código, nombre, latitud y longitud de una parada.
NodoTrie	Nodo del árbol de prefijos. Guarda un carácter, un arreglo fijo de 256 hijos y, si marca fin de palabra, el nombre completo y el código asociado.
Trie	Árbol de prefijos. Inserta nombres de parada y recolecta sugerencias a partir de un prefijo.
NodoHash	Nodo de la lista enlazada de un cubo. Guarda clave, valor y referencia al siguiente nodo.
TablaHash	Tabla hash genérica con encadenamiento separado. Inserta y busca por clave.
MinHeap	Cola de prioridad mínima sobre dos arreglos paralelos (vértice y prioridad). Es la cola de Dijkstra.
NodoArista	Nodo de la lista de adyacencia. Guarda destino, peso y referencia a la siguiente arista.
RutaResultado	Contenedor del resultado de Dijkstra: existencia de ruta, distancia total, cantidad de paradas, códigos, paradas y mensaje.
Grafo	Grafo ponderado con listas de adyacencia. Ofrece Dijkstra, BFS, DFS y el cálculo de distancia de Haversine.
UnionFind	Conjuntos disjuntos con compresión de ruta y unión por rango. Responde si dos paradas están conectadas.
LectorCSV	Clase estática que carga paradas y conexiones, y sincroniza Trie, Tabla Hash, Grafo y Union-Find.
FormPrincipal	Ventana principal. Contiene las instancias de las estructuras y orquesta el flujo A, B, C, D.
Program	Punto de entrada. Habilita los estilos visuales y ejecuta FormPrincipal.

5. Diagramas de flujo

La Figura 4 describe el flujo general de uso de la aplicación, desde el inicio hasta la presentación de la ruta. La Figura 5 detalla el algoritmo de Dijkstra tal como está implementado, y la Figura 6 muestra los recorridos BFS y DFS.

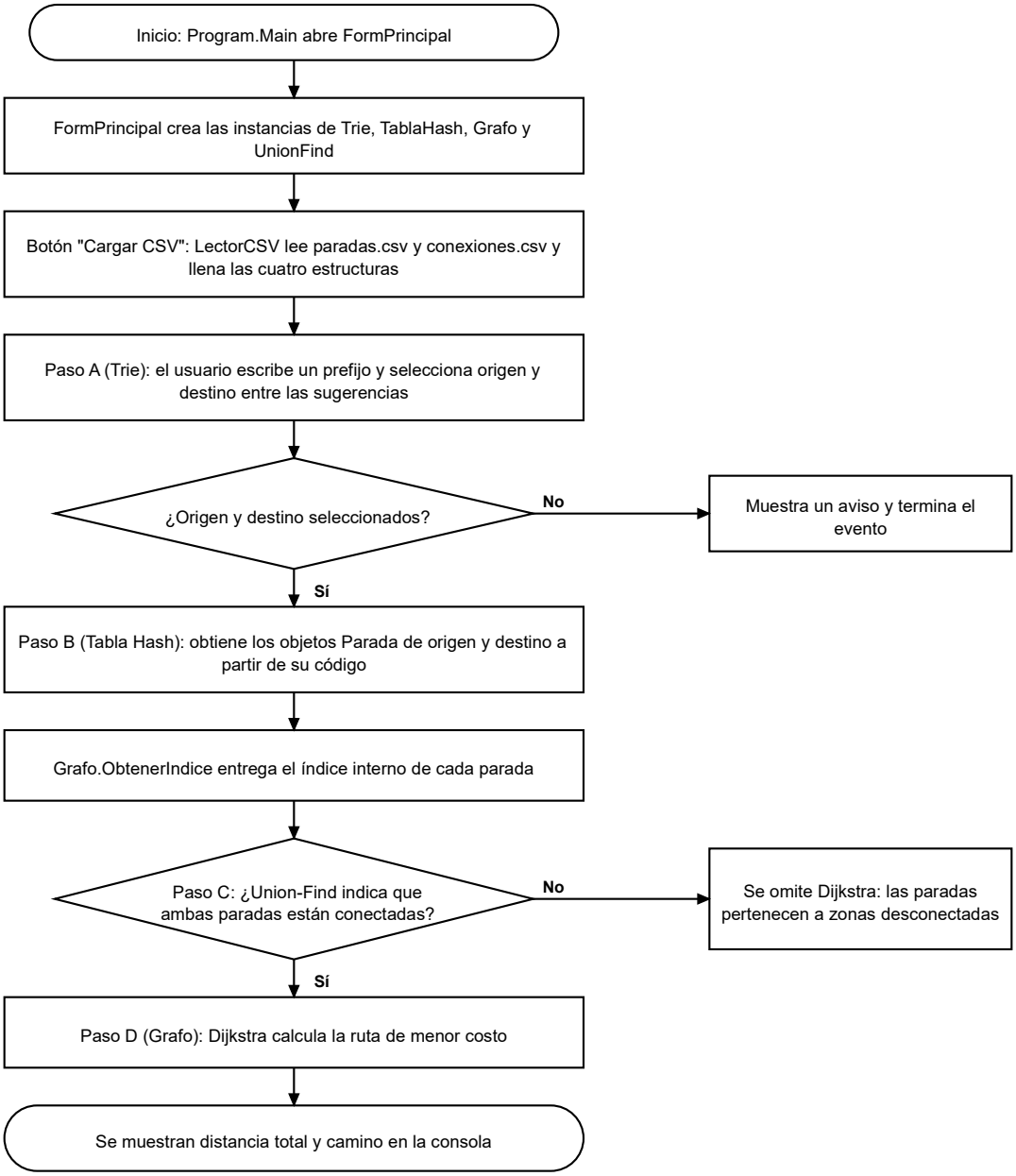


Figura 4. Diagrama de flujo general del sistema.

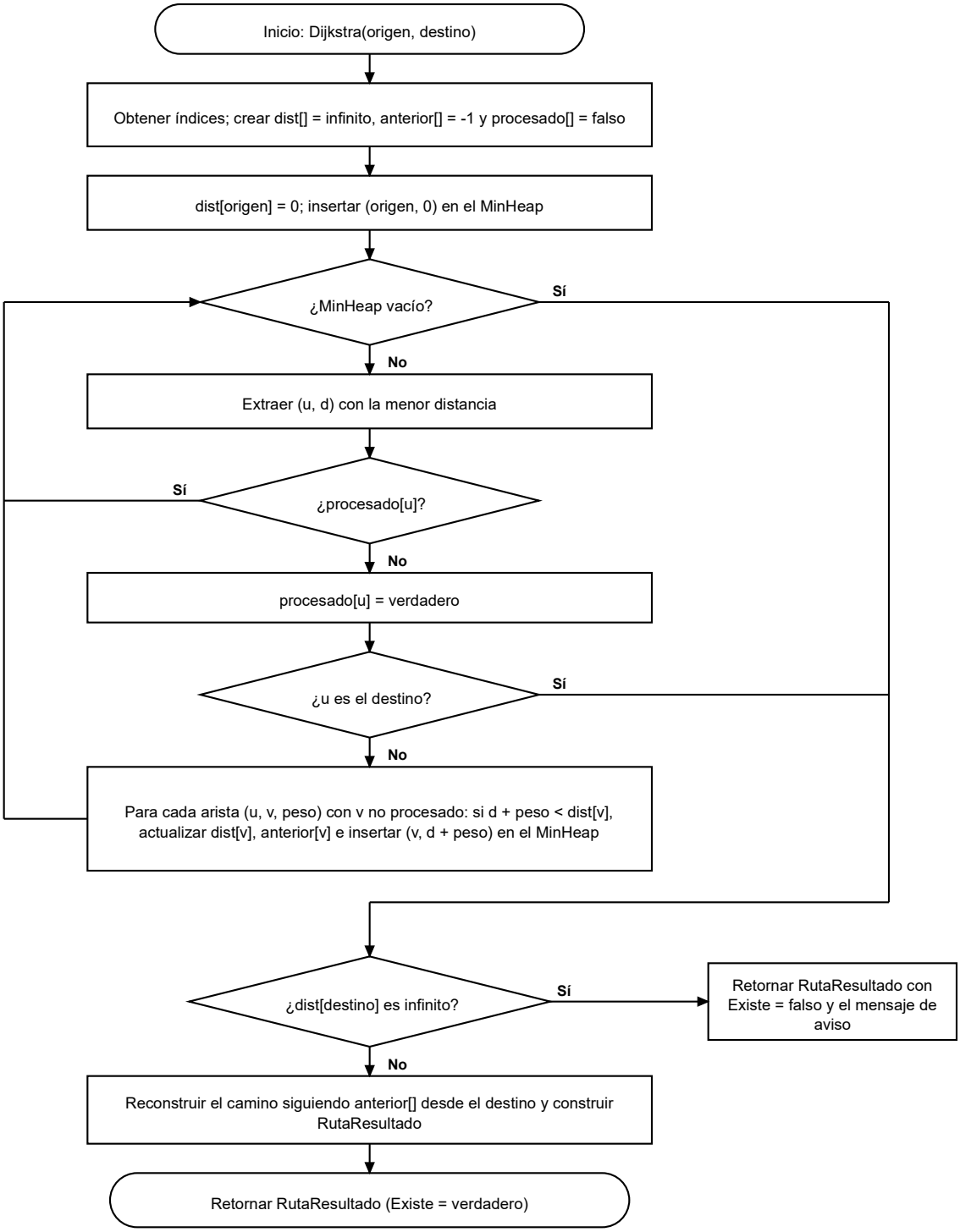


Figura 5. Diagrama de flujo del algoritmo de Dijkstra.

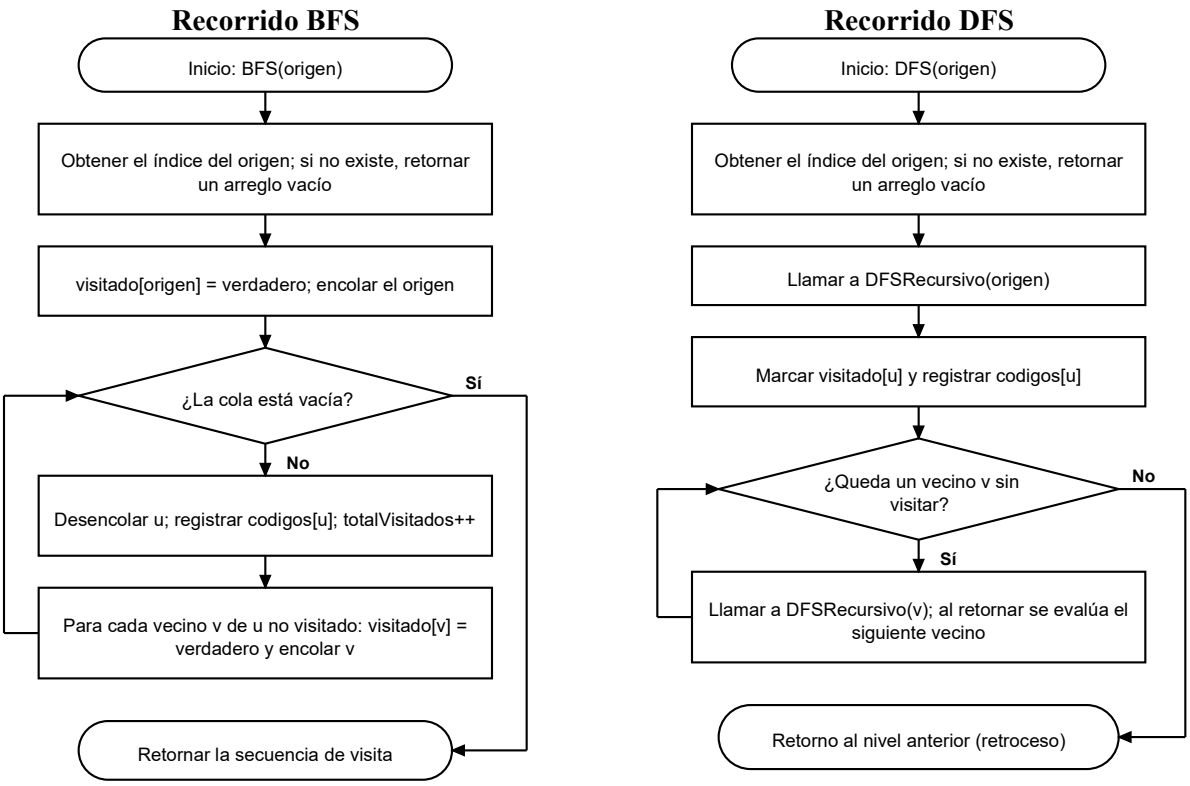


Figura 6. Diagramas de flujo de los recorridos BFS y DFS.

6. Explicación de la implementación

6.1 Clase Parada

Parada es la entidad básica del sistema. Sus cuatro campos públicos (Codigo, Nombre, Latitud y Longitud) se asignan en el constructor. El método MostrarInformacion imprime los datos en la consola. El código es el identificador único: el Trie, la Tabla Hash y el Grafo lo utilizan para referirse a la misma parada sin duplicar el objeto.

6.2 Árbol de prefijos (Trie)

6.2.1 Estructura

Cada NodoTrie representa un carácter. Su campo Hijos es un arreglo de 256 referencias, indexado por el código numérico del carácter, por lo que el hijo correspondiente a una letra se obtiene sin búsqueda. Un nodo con EsFinDePalabra en verdadero marca el final de un nombre y guarda en PalabraCompleta el nombre completo y en CodigoAsociado el código de la parada. La raíz es un nodo vacío (carácter nulo) y no representa ningún símbolo del nombre.

6.2.2 Inserción

El método Insertar recorre el nombre carácter por carácter desde la raíz. Si el hijo del carácter actual no existe, lo crea; luego desciende a él. Al terminar, marca el último nodo como fin de palabra y almacena el nombre y el código. Los caracteres cuyo código es mayor o igual que 256 se omiten.

6.2.3 Autocompletado

ObtenerSugerencias desciende por el árbol siguiendo los caracteres del prefijo. Si en algún punto falta el hijo, no existen coincidencias y el método termina. Al llegar al nodo del último carácter del prefijo, RecolectarRecursivo recorre en profundidad el subárbol y agrega al arreglo de sugerencias cada nodo que marca fin de palabra. El recorrido se detiene cuando el contador alcanza el límite máximo (10 en la interfaz). Como los hijos se visitan de la posición 0 a la 255, las sugerencias resultan ordenadas por el código de los caracteres. Los resultados se devuelven mediante parámetros ref, sin usar listas dinámicas.

6.3 Tabla Hash

La tabla es genérica en clave y valor. Internamente mantiene un arreglo de M cubos (101 por defecto, un número primo que reduce las colisiones). Cada cubo es el inicio de una lista enlazada de NodoHash.

La función hash es polinómica con base 31: se recorre el texto de la clave y en cada paso se calcula hash = (hash * 31 + c) % capacidad. La reducción módulo M en cada paso evita el desbordamiento del entero. Las colisiones se resuelven por encadenamiento separado: dos claves con el mismo índice comparten el cubo y se enlazan en una lista.

Insertar calcula el índice, recorre la lista del cubo y, si encuentra la clave, reemplaza el valor; si no, agrega un nodo al final. Buscar repite el recorrido y devuelve el valor, o el valor por defecto del tipo (0 para enteros y null para objetos) cuando la clave no existe.

La tabla se utiliza en dos lugares: TablaHash<string, Parada> en FormPrincipal para recuperar paradas por código, y TablaHash<string, int> dentro del Grafo para traducir un código en un índice de arreglo.

6.4 Grafo y algoritmos de caminos óptimos

6.4.1 Representación

Cada parada recibe un índice entero consecutivo (0, 1, 2, ...). Los datos de un vértice se guardan en la misma posición de cuatro arreglos paralelos: codigos (código), paradas (objeto Parada), cabezas (primera arista de la lista de adyacencia) y colas (última arista). Mantener el puntero a la cola permite insertar una arista nueva en tiempo constante, sin recorrer la lista. La Tabla Hash indices traduce el código de una parada en su índice.

Un detalle de diseño importante es que el índice se almacena en la tabla sumándole uno. Como Buscar devuelve 0 cuando la clave no existe, almacenar el índice real confundiría al vértice 0 con un resultado inexistente. ObtenerIndice resta uno al consultar, de modo que devuelve -1 cuando el código no está registrado.

Cuando los arreglos se llenan, CrecerArreglos crea arreglos del doble de tamaño y copia los elementos. La duplicación hace que el costo amortizado de agregar un vértice sea constante.

6.4.2 Construcción: vértices y aristas

AgregarVertice rechaza códigos duplicados, asigna el siguiente índice libre y registra el vértice. AgregarArista resuelve los índices de origen y destino y, si alguno no existe, devuelve falso. Si no se indica peso, lo calcula con la distancia de Haversine entre las coordenadas de ambas paradas. En un grafo no dirigido inserta el arco en los dos sentidos mediante PonerArco.

6.4.3 Cola de prioridad MinHeap

El montículo mínimo es un árbol binario completo almacenado en un arreglo, en el que cada nodo es menor o igual que sus hijos. Para el nodo en la posición i, el padre está en (i - 1) / 2 y los hijos en 2i + 1 y 2i + 2. La raíz, en la posición 0, siempre contiene la menor prioridad. Se emplean dos arreglos paralelos: vertices y prioridades.

- Insertar coloca el elemento en la primera posición libre y lo hace "subir" intercambiándolo con su padre mientras sea menor.
- ExtraerMin devuelve la raíz, mueve el último elemento a la posición 0 y lo hace "bajar" intercambiándolo con el menor de sus hijos hasta restablecer el orden.
- Crecer duplica la capacidad de ambos arreglos cuando se llenan.
- Intercambiar intercambia siempre ambos arreglos para que vértice y prioridad no se desincronicen.

6.4.4 Algoritmo de Dijkstra

Dijkstra calcula la ruta de menor costo entre un origen y un destino en un grafo con pesos no negativos. La idea es fijar, en cada paso, el vértice no procesado con menor distancia conocida y usarlo para mejorar las distancias de sus vecinos (relajación). La implementación utiliza tres arreglos de tamaño V: dist (mejor distancia conocida), anterior (predecesor en el mejor camino) y procesado (indica si la distancia ya es definitiva).

1. Inicialización: todas las distancias en infinito, todos los predecesores en -1, y dist[origen] = 0.
   Se inserta el origen en el MinHeap con prioridad 0.
2. Se extrae del montículo el par (u, d) de menor distancia. Si u ya fue procesado, el par es una entrada obsoleta y se descarta.

3. Se marca u como procesado. Si u es el destino, el ciclo termina anticipadamente, porque su distancia ya es mínima.
4. Para cada arista (u, v, peso) con v no procesado, se calcula d + peso. Si es menor que dist[v], se actualizan dist[v] y anterior[v] y se inserta (v, d + peso) en el montículo.
5. Si dist[destino] sigue en infinito, no existe camino y se devuelve un resultado con Existe en falso y un mensaje.
6. En caso contrario se reconstruye el camino: primero se cuentan los vértices siguiendo anterior desde el destino, luego se llena el arreglo del camino de atrás hacia adelante y finalmente se convierten los índices en códigos y objetos Parada.

El montículo no admite la operación de disminuir una clave existente. En su lugar, cada mejora inserta una entrada nueva y la entrada anterior queda obsoleta; se descarta al extraerla gracias a la verificación de procesado. Esta técnica, llamada eliminación perezosa, simplifica la implementación sin alterar la corrección del algoritmo.

6.4.5 Recorrido en anchura (BFS)

BFS visita primero los vecinos directos del origen, después los vecinos de estos, y así por niveles. Utiliza una cola FIFO (Queue<int>) y un arreglo visitado. El origen se marca como visitado al encolarlo, y no al desencolarlo, para impedir que un vértice entre más de una vez en la cola. El método devuelve un arreglo de cadenas con los códigos en orden de visita y, mediante el parámetro out totalVisitados, la cantidad de posiciones válidas: el arreglo tiene tamaño V, pero solo las primeras totalVisitados posiciones contienen datos.

6.4.6 Recorrido en profundidad (DFS)

DFS avanza por un camino hasta donde es posible antes de retroceder y probar otro. Se implementa de forma recursiva en DFSRecursivo: marca el vértice, lo registra y llama recursivamente para cada vecino no visitado. El contador se comparte entre las llamadas mediante el parámetro ref totalVisitados. La recursión utiliza la pila de llamadas del sistema, por lo que no requiere una estructura adicional, pero su profundidad máxima es V.

6.5 Union-Find

Union-Find mantiene una partición de los vértices en conjuntos disjuntos, cada uno correspondiente a una zona conectada de la red. Usa dos arreglos: padre (representante del padre de cada elemento; una raíz es su propio padre) y rango (cota superior de la altura del árbol). Sus índices coinciden con los índices internos del Grafo.

- Find: sigue los punteros hasta la raíz y, al regresar de la recursión, apunta cada nodo visitado directamente a la raíz (compresión de ruta). Las consultas posteriores son casi inmediatas.
- Union: busca las raíces de ambos elementos. Si son distintas, cuelga la raíz de menor rango bajo la de mayor rango (unión por rango). Si los rangos son iguales, elige una raíz y aumenta su rango en uno.
- EstanConectados: compara las raíces de dos elementos. Dos paradas están conectadas si y solo si tienen la misma raíz.
- AsegurarCapacidad: duplica los arreglos cuando el Grafo agrega vértices por encima de la capacidad actual, inicializando las posiciones nuevas como conjuntos unitarios.

Durante la carga de conexiones, cada arista creada en el Grafo se refleja con una operación Union entre los índices de sus extremos. Al terminar la carga, los conjuntos coinciden con las componentes conexas del grafo. La consulta del Paso C permite rechazar una ruta imposible sin ejecutar Dijkstra, que en ese caso recorrería toda la componente del origen para concluir que no hay camino. Esta equivalencia es válida porque el grafo es no dirigido; en un grafo dirigido la pertenencia al mismo conjunto no garantizaría alcanzabilidad en un sentido concreto.

6.6 Lector de archivos CSV

CargarParadas lee el archivo completo con File.ReadAllLines, ignora el encabezado y las líneas vacías, separa los campos con Split(',') y convierte las coordenadas con cultura invariante. Cada parada válida se inserta en las cuatro estructuras: Trie, Tabla Hash, Grafo (como vértice) y Union-Find (se asegura su capacidad). CargarConexiones procesa cada línea del archivo de conexiones, calcula el peso opcional y, si la arista se agrega al Grafo, une los índices en Union-Find. Ambos métodos devuelven un mensaje con la cantidad de registros cargados o un aviso si el archivo no existe.

6.7 Interfaz gráfica

La ventana FormPrincipal se divide en dos archivos mediante la palabra clave partial. El archivo del diseñador crea los controles y asigna los eventos en ConfigurarVentana; el archivo de lógica contiene los manejadores:

- BtnCargarDatos_Click: invoca los dos métodos de carga del LectorCSV y muestra los mensajes resultantes en la consola.
- BuscarSugerencias: se ejecuta cada vez que cambia el texto de un cuadro de búsqueda. Solicita al Trie hasta 10 sugerencias y las muestra con el formato "Nombre [Código]".
- SeleccionarParada: extrae el código que aparece entre corchetes en el elemento elegido y lo guarda como origen o destino (Paso A).
- BtnCalcularRuta_Click: valida que haya origen y destino, ejecuta los pasos B, C y D, y escribe distancia total y camino en la consola.
- BtnEjecutarBFS_Click y BtnEjecutarDFS_Click: ejecutan los recorridos desde el origen seleccionado y muestran la secuencia de visita.
- Log: agrega una línea al cuadro de texto de solo lectura y desplaza la vista al final.

7. Análisis de complejidad (Big-O)

Esta sección presenta el costo en tiempo y espacio de cada operación, y justifica la elección de cada estructura comparándola con alternativas razonables. La notación está definida en la sección 1.4.

7.1 Resumen de complejidades

Estructura	Operación	Caso promedio	Peor caso	Espacio
Trie	Insertar	O(m)	O(m)	O(T · Sigma), con T = total de nodos
Trie	ObtenerSugerencias	O(m + k · L)	O(m + Sigma · S), con S = nodos del subárbol del prefijo	O(1) adicional, más la pila de recursión O(L)
Tabla Hash	Insertar / Buscar	O(1 + n / M)	O(n)	O(n + M)
MinHeap	Insertar	O(log n)	O(log n)	O(n)
MinHeap	ExtraerMin	O(log n)	O(log n)	O(1) adicional
Grafo	AgregarVertice	O(1) amortizado	O(V) al duplicar arreglos	O(V + E)
Grafo	ObtenerIndice	O(1)	O(V)	O(1)
Grafo	AgregarArista	O(1)	O(1)	O(1) por arco
Grafo	Dijkstra	O((V + E) log V)	O((V + E) log V)	O(V + E)
Grafo	BFS	O(V + E)	O(V + E)	O(V)
Grafo	DFS	O(V + E)	O(V + E)	O(V), incluida la pila de recursión
Union-Find	Find / Union	O(alfa(n)) amortizado	O(log n) en una operación aislada	O(n)
Union-Find	EstanConectados	O(alfa(n)) amortizado	O(log n)	O(1) adicional
LectorCSV	CargarParadas	O(n · m)	$O(n \cdot m + n^2)$ con colisiones extremas	O(n · m)
LectorCSV	CargarConexiones	O(E · alfa(V))	O(E · log V)	O(E)

Notas: el costo de ObtenerSugerencias incluye el factor constante de recorrer las 256 posiciones de cada nodo visitado. El peor caso de Union-Find, O(log n), se debe a que la unión por rango acota la altura de los árboles; la compresión de ruta lleva el costo amortizado a O(alfa(n)).

7.2 Justificación de las estructuras de búsqueda

7.2.1 Trie frente a otras alternativas para el autocompletado

El autocompletado requiere encontrar todos los nombres que comienzan con un prefijo. El Trie resuelve el descenso al prefijo en O(m), independiente del número de paradas, y las sugerencias están agrupadas en un solo subárbol. Una tabla hash no sirve para esta tarea, porque dispersa las claves y no conserva relación entre prefijos. Un arreglo ordenado permite localizar el prefijo con búsqueda binaria, pero insertar exige desplazar elementos.

Alternativa	**Búsqueda por prefijo**	**Inserción**	**Observación**
Trie (elegida)	$O(m + k \cdot L)$	$O(m)$	Costo independiente de n. Mayor consumo de memoria.
Arreglo sin ordenar con filtrado	$O(n \cdot m)$	$O(1)$	Simple, pero debe revisar todas las paradas.
Arreglo ordenado con búsqueda binaria	$O(m \log n + k)$	$O(n)$	Inserción costosa por desplazamiento de elementos.
Árbol balanceado (AVL)	$O(m \log n + k)$	$O(m \log n)$	Cada comparación de cadenas cuesta O(m). Rotaciones complejas.
Tabla hash	$O(n \cdot m)$	$O(1)$	No admite consulta por prefijo.

La contrapartida del Trie es la memoria. Con un arreglo fijo de 256 referencias por nodo, cada nodo ocupa del orden de 2 KB en una arquitectura de 64 bits. Los diez nombres del conjunto de ejemplo suman unos 130 nodos, es decir, cerca de 270 KB. El diseño se considera adecuado para un catálogo de paradas de tamaño moderado, y su ventaja es que cada paso del descenso es un acceso directo por índice, sin comparaciones.

7.2.2 Tabla Hash frente a otras alternativas para la búsqueda por código

Una vez seleccionada una sugerencia se conoce el código exacto de la parada, y solo se necesita recuperar el objeto asociado. La búsqueda por clave exacta, sin necesidad de orden, es el caso para el que la tabla hash está diseñada. Con un factor de carga bajo (10 paradas en 101 cubos, aproximadamente 0.1), las listas de cada cubo tienen una longitud media muy inferior a uno.

Alternativa	**Búsqueda**	**Inserción**	**Observación**
Tabla hash (elegida)	O(1) promedio, O(n) peor caso	O(1) promedio	Ideal para claves exactas. No conserva orden.
Arreglo ordenado con búsqueda binaria	$O(\log n)$	$O(n)$	Requiere mantener el orden en cada inserción.
Lista enlazada	$O(n)$	$O(1)$	Búsqueda lineal.

Árbol balanceado (AVL)	O(log n)	O(log n)	Garantiza el peor caso, pero con mayor complejidad de código.

La implementación actual usa un tamaño fijo de 101 cubos y no redimensiona. Mientras n sea del orden de M, el costo promedio es constante; si n crece mucho más que M, las listas se alargan y el costo tiende a O(n / M), que asintóticamente es O(n). Para catálogos grandes convendría duplicar los cubos y redistribuir las claves cuando el factor de carga supere un umbral.

7.3 Justificación del grafo y de los algoritmos de rutas

7.3.1 Lista de adyacencia frente a matriz de adyacencia

Una red de transporte es un grafo disperso: cada parada se conecta con unas pocas vecinas, de modo que E es del orden de V y muy inferior a V^2. En el conjunto de ejemplo hay 10 vértices y 9 aristas (18 arcos); una matriz necesitaría 100 celdas, de las cuales 82 estarían vacías. La lista de adyacencia almacena solo las conexiones existentes y permite recorrer los vecinos de un vértice en proporción a su grado.

Criterio	**Lista de adyacencia (elegida)**	**Matriz de adyacencia**
Espacio	O(V + E)	$O(V^2)$
Recorrer los vecinos de u	O(grado de u)	O(V)
Consultar si existe la arista (u, v)	O(grado de u)	O(1)
Agregar una arista	O(1) con puntero a la cola	O(1)
BFS o DFS completo	O(V + E)	$O(V^2)$
Dijkstra con MinHeap	O((V + E) log V)	$O(V^2)$ con montículo, $O(V^2)$ con arreglo

La única operación en que la matriz es superior es la consulta de existencia de una arista concreta, que el sistema no necesita: los algoritmos implementados recorren vecinos, no consultan aristas puntuales.

7.3.2 MinHeap frente a otras colas de prioridad

Dijkstra realiza una inserción por cada mejora de distancia y una extracción por cada entrada de la cola. La elección de la cola determina el costo total del algoritmo.

Estructura	**Insertar**	**Extraer mínimo**	**Observación**
MinHeap binario (elegida)	O(log n)	O(log n)	Sencillo, sobre arreglo, constantes bajas.
Arreglo sin ordenar	O(1)	O(n)	Dijkstra total $O(V^2)$. Adecuado solo en grafos densos.
Arreglo ordenado	O(n)	O(1) si se extrae del extremo	Inserción costosa por desplazamiento.

Lista enlazada ordenada	O(n)	O(1)	Inserción lineal.
Árbol balanceado	O(log n)	O(log n)	Mayor memoria y complejidad sin ganancia asintótica.
Heap de Fibonacci	O(1) amortizado	O(log n) amortizado	Dijkstra O(E + V log V), con constantes altas y gran complejidad de implementación.

7.3.3 Complejidad de Dijkstra

La inicialización de los tres arreglos cuesta O(V). Con eliminación perezosa, cada arco produce a lo sumo una inserción en el montículo (más la inserción inicial del origen), de modo que el montículo contiene como máximo O(E) entradas y se realizan O(E) inserciones y O(E) extracciones, cada una de costo O(log2E).
La finalización anticipada al extraer el destino no mejora el peor caso, pero reduce el trabajo real cuando el destino está cerca del origen.

Algoritmo	**Tiempo**	**Pesos admitidos**	**Resultado**	**Razón de descarte o uso**
Dijkstra con MinHeap (elegido)	O((V + E) log V)	No negativos	Un origen a todos	Mejor opción para grafos dispersos con pesos positivos.
Dijkstra con arreglo lineal	$O(V^2)$	No negativos	Un origen a todos	Preferible solo si el grafo es denso (E cercano a V^2).

7.3.4 Complejidad de BFS y DFS

Ambos recorridos marcan cada vértice una sola vez y examinan cada arco una vez al recorrer la lista de adyacencia de su origen, por lo que el tiempo es O(V + E) en todos los casos. Los dos exigen espacio O(V) para el arreglo de visitados y el de resultados; BFS añade la cola, que contiene como máximo V vértices, y DFS añade la pila de recursión, cuya profundidad máxima es V.

Variante	**Tiempo**	**Espacio**	**Observación**
BFS con cola (elegida)	O(V + E)	O(V)	Visita por niveles. En un grafo no ponderado encuentra caminos con el menor número de aristas.
DFS recursivo (elegida)	O(V + E)	O(V) de pila	Código breve. Con redes muy profundas podría agotar la pila de llamadas.
DFS iterativo con pila explícita	O(V + E)	O(V)	Evita el desbordamiento de pila; requiere implementar la pila manualmente.
Recorridos sobre matriz de adyacencia	$O(V^2)$	$O(V^2)$	Examinan V posiciones por vértice aunque no haya aristas.

Los recorridos cumplen un propósito distinto al de Dijkstra: describen la estructura de la red, es decir, qué paradas son alcanzables y en qué orden, sin considerar los pesos. Por esta razón no son alternativas intercambiables, sino herramientas complementarias.

7.4 Justificación de Union-Find

La pregunta "¿pertenecen estas dos paradas a la misma zona conectada?" se formula antes de cada cálculo de ruta y debe responderse rápido, mientras las conexiones se incorporan durante la carga de datos. Union-Find permite ambas cosas: unir conjuntos al leer cada arista y consultar la pertenencia en tiempo casi constante.

Alternativa	**Consulta**	**Incorporar una arista**	**Observación**
Union-Find con rango y compresión (elegida)	O(alfa(n)) amortizado	O(alfa(n)) amortizado	Incremental y de espacio O(n). No permite eliminar aristas.
Quick-Find (arreglo de identificadores)	O(1)	O(n)	La unión recorre todo el arreglo.
Quick-Union sin optimizaciones	O(n) peor caso	O(n) peor caso	Los árboles pueden degenerar en listas.
BFS o DFS en cada consulta	O(V + E)	O(1)	Repite el recorrido completo por cada consulta.

El beneficio concreto en el sistema es evitar ejecutar Dijkstra cuando no puede haber ruta. La consulta cuesta O(alfa(n)), frente a O((V + E) log V) del algoritmo de rutas, que en un grafo desconectado exploraría toda la componente del origen antes de concluir que no existe camino.

7.5 Costo de una consulta de ruta completa

El costo de las acciones del usuario, de la escritura de un prefijo a la presentación del resultado, se resume así:

Etapa	**Operación**	**Costo**
A	Autocompletado del origen y del destino (por cada cambio de texto)	O(m + k · L)
B	Dos búsquedas en la Tabla Hash	O(1) promedio
C	Dos índices con ObtenerIndice y una consulta de Union-Find	O(1) + O(alfa(n))
D	Dijkstra	O((V + E) log V)
Total	Predomina la ejecución de Dijkstra	O((V + E) log V)

8. Limitaciones conocidas y mejoras propuestas

La revisión del código permitió identificar los siguientes puntos. Ninguno afecta el funcionamiento con el conjunto de datos de ejemplo, pero conviene tenerlos presentes al ampliar el sistema.

Aspecto	Situación actual	Mejora propuesta
Búsqueda en el Trie	Distingue mayúsculas, minúsculas y acentos: el prefijo "estacion" no sugiere "Estacion Central". Los caracteres con código mayor o igual a 256 se omiten.	Normalizar el texto (minúsculas y sin acentos) antes de insertar y de buscar.
Tamaño de la Tabla Hash	Número fijo de 101 cubos, sin redimensión.	Redimensionar y redistribuir cuando el factor de carga supere un umbral.
DFS recursivo	La profundidad de recursión puede llegar a V y agotar la pila en redes muy extensas.	Reescribir con una pila explícita.
Validación en Dijkstra	No comprueba que origen y destino existan; un índice -1 produciría un error. La interfaz lo evita porque los códigos provienen del Trie.	Validar los índices al inicio y devolver un resultado con mensaje.
Pesos negativos	El archivo de conexiones admite cualquier número, pero Dijkstra solo es correcto con pesos no negativos.	Rechazar pesos negativos durante la carga.
Aristas repetidas	Una conexión duplicada en el archivo se agrega dos veces. No afecta el resultado de Dijkstra.	Verificar la existencia de la arista antes de insertarla.
Resultado sin ruta	La interfaz no muestra un mensaje cuando Dijkstra devuelve Existe en falso; lo evita el filtro de Union-Find.	Registrar el mensaje de RutaResultado en la consola.
Grafo dirigido	Union-Find equivale a conectividad solo en grafos no dirigidos, que es la configuración usada.	Usar componentes fuertemente conexas si se adoptan sentidos únicos.
Nombres con coma	El separador del CSV impide nombres que contengan comas.	Admitir campos entre comillas.

9. Conclusiones

El sistema combina cuatro estructuras de datos implementadas manualmente, cada una elegida por el tipo de consulta que debe resolver: el Trie para búsquedas por prefijo, la Tabla Hash para acceso por código exacto, Union-Find para consultas de conectividad y el grafo con lista de adyacencia para representar una red dispersa. El algoritmo de Dijkstra con cola de prioridad binaria ofrece el mejor equilibrio entre eficiencia y simplicidad para calcular rutas con pesos positivos, y los recorridos BFS y DFS, con costo lineal, permiten explorar la red.

El flujo de cuatro pasos reduce el trabajo innecesario: la conectividad se verifica en tiempo casi constante antes de recurrir al algoritmo de mayor costo. Las limitaciones señaladas en la sección 9 son extensiones naturales del diseño y no cambian su estructura.